

ELEC 475 Lab 1

Model Details

Using *PyTorch*, this model implements a 4-layer MLP autoencoder to compress input data to a lower dimension by encoding and reconstructs it by decoding.

The class *autoencoderMLP4Layer* utilizes *PyTorch*'s neural network library, *nn.module*.

Initialization

The `__init__` method is set up with parameters controlling the size of the input, output, and bottleneck. We give it an input size of 784 (28x28), which will be enough to represent our *MNIST* images. Then, the bottleneck is allotted 8 dimensions. This value is significant because it reduces the dimensionality significantly, while still allowing for accurate reconstruction. The output is set the same as the input, so it can reconstruct in the original size. `N2` is another parameter hardcoded as 392 within the method body, defining a fixed size for the intermediate layers. Then, we create the layers *fc1*, *fc2*, *fc3*, and *fc4*, using the previously defined sizes to encode and decode the image, with each layer being fully connected.

Forward Pass

The *forward* method is used to define how the images will flow through the model. Its input parameter *X* is set up to be a *PyTorch* tensor of shape (*batch_size*, *N_input*). The method adjusts the size of *X* by calling the four layers defined in `__init__`, and calling the *ReLU* activation function after each layer to form the compressed representation. The images are encoded going to *fc1* and *fc2*, and decoded from *fc3* and *fc4*. Lastly, the sigmoid activation is applied to *X* to scale the outputs within the range [0,1], which is in line with the *MNIST* images.

Training Details

Model Configuration

The training model utilizes the *autoencoderMLP4Layer* class. The model takes an input size of 784 (28 x 28 *MNIST* images flattened), a bottleneck size of 8, and an output size of 784 (same as input for reconstruction).

It is then trained on the *MNIST* training dataset which is first transformed using *transforms.ToTensor()* and preprocessed/flattened using *imgs.view(imgs.shape[0], -1)*.

Then the data is optimized using *Adam* (Adaptive Moment Estimation) with a learning rate of 1e-3 and a weight decay of 1e-5. It then calls the *ReduceLROnPlateau()* scheduler function with mode set to min to reduce the learning rate when training loss plateaus. It then computes the *MSE* (Mean Squared Error) Loss with reduction equal to mean.

The weights are initialized using *Xavier* uniform initialization with a bias of 0.01.

Lastly, before training the model, it will detect if CUDA is available as it will train much faster using GPU, otherwise it will train using CPU.

Command Line Parameters

As for the training parameters, they are defined in the command line like this:

```
python train.py -z 8 -e 50 -b 2048 -s MLP.8.pth -p loss.MLP.8.png
```

This executes the *train.py* file with a bottleneck size (-z) of 8, epoch size (-e) of 50, batch size (-b) of 2048, saves (-s) the file in *MLP.8.pth*, and plots (-p) the training loss in *loss.MLP.8.png*.

Process

For each epoch:

- Iterate through all batches in training set
- Forward pass: compute outputs
- Compute *MSE* loss between input/output images
- Backward pass: compute gradients using *loss.backward()*
- Updates params using *optimizer.step()*
- Clears gradients using *optimizer.zero_grad()*
- Updates learning rate scheduler based off epoch loss

Results

Step 3 Training

Below is our *loss.MLP.8.png* that represents a well-behaved loss curve plot, confirming that the model trains correctly.

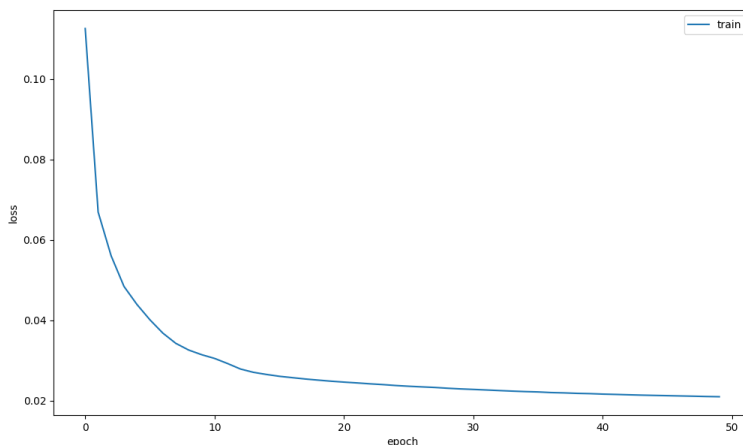


Figure 1: Loss curve plot with epoch on x-axis and loss on y-axis.

The model is improving its prediction accuracy as epochs increase. There is an initial steep decline where the weights are being rapidly adjusted between 0-5 epochs, but then gradually plateaus at around 15 epochs. This indicates that it is converging on an optimal solution and has found a good approximation of weights.

Step 4 Output

The autoencoder showcases its ability to compress and reconstruct images in the form of digits.

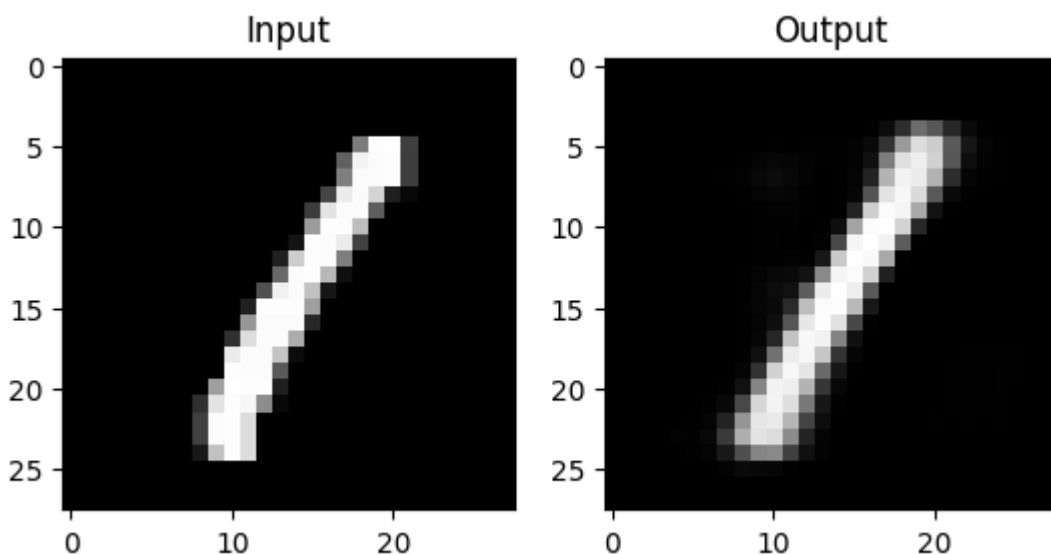


Figure 2: Comparison of the original digit and its reconstructed image.

The reconstructed digit was able to keep the same essential features as the original.

Step 5 Output

This step involves adding noise to the images and using the autoencoder to reconstruct them without noise. The result shows that it can effectively denoise the digit.

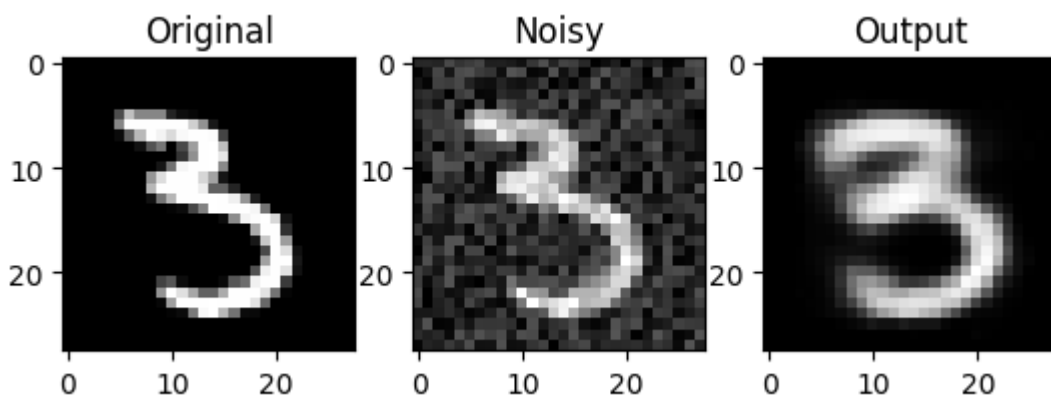


Figure 3: Comparison of the original, noisy, and denoised images.

Step 6 Output

In this step, we demonstrated bottleneck interpolation with the autoencoder. It was successful in blending the features of two randomly selected images from the *MNIST* dataset. After encoding them into their bottleneck representation, linear interpolation is used to generate a series of n new intermediate bottleneck tensors. As seen below, the bottleneck layer effectively captures and encodes key features of the digits.

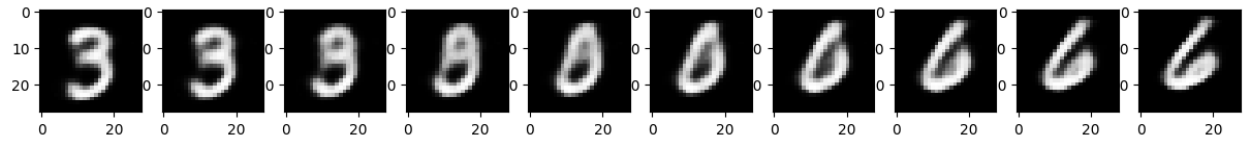


Figure 4: Images of the bottleneck interpolation process.